CAREER ROADMAP · FULL STACK ENGINEER

The ₹35 LPA Roadmap

From Delta Electronics to SDE-2 at a top Indian product company

For: Santhosh R **Prepared:** September 2026 **Target window:** May – August 2027 **Experience then:** ~3.5 years

1. What ₹35 LPA Actually Means

₹35 LPA at three-and-a-half years is **SDE-2 at a tier-1 Indian product company**. It is reachable from where you are, but it is not the median outcome — it is roughly the 80th percentile of that cohort. Getting there is a function of four things, and only one of them is on your resume today.

Component	Typical share	What it means for you
Fixed base	₹24 – 28 L	The number that pays your bills. Negotiate this hardest.
ESOP / RSU (annualised)	₹4 – 8 L	Startups quote 4-year totals to inflate the headline. Always divide by 4.
Performance variable	₹2 – 4 L	Usually 10% of base, paid at 80–100%. Treat 90% as the realistic figure.
Joining bonus	₹1 – 3 L	One-time. Never let it be counted as "CTC".

READ EVERY OFFER THIS WAY

A ₹35 L "CTC" that is ₹19 L base + ₹40 L ESOP over 4 years + ₹3 L joining is really **₹19 L of guaranteed money**. A ₹31 L offer at ₹27 L base is the better job. Ask for the written breakup before you react to any number.

2. Where You Stand Today — Honest Scorecard

Dimension	Today	Evidence / What's missing
Product ownership	STRONG	Sole developer of a 3-app platform shipped on customer hardware. Better scope than most 2.5-yr devs.
Frontend (React/TS)	STRONG	Production React 18/19, Redux, component library from Figma, hardware-constrained UI, React Native app.
Auth & API design	STRONG	JWT access + httpOnly refresh, permissions re-resolved on refresh, 17 route modules, OpenAPI. This is real.
Linux / deployment	STRONG	systemd, nginx, udev, ARM64 packaging, OTA + rollback. Rare and undervalued — reframe it as DevOps.
DSA	BLOCKING	No evidence at all. Every target company gates on 2 DSA rounds *before* anyone reads the above.
Scale & metrics	BLOCKING	Zero numbers in the resume. No users, RPS, latency, or data volume. Reads as "one device, one user".
Distributed systems	BLOCKING	No Redis, queues, cloud, containers-in-prod, CI/CD, or observability. Backend is one Node process on a box.
System design	IN PROGRESS	Curriculum underway. Needs to shift toward LLD, which is 70% of what a 3-yr candidate is actually asked.
Domain narrative	RISK	Industrial IoT → consumer/fintech is a story you must actively bridge, or recruiters deprioritise the profile.
Network / brand	WEAK	No referral pipeline, no public writing. Cold applications from a hardware company get filtered out.

THE CORE INSIGHT

Your engineering *depth* is already above your years. The problem is that it is **invisible** (no numbers on anything) and **unproven** (no DSA, no scale). Both are fixable in about eight months. Neither requires abandoning what you have built — the plan below converts your existing embedded/Linux work into cloud vocabulary rather than replacing it.

ONE TITLE RISK TO MANAGE

"Senior Software Engineer" at 2.5 years is Delta's hardware-industry title scale. Product-company recruiters read it as inflated and it can trigger distrust or the wrong (higher) bar. Keep the title on the resume — it is real — but in outreach and interviews always lead with *years and scope*, never with the word "senior".

3. The Eight-Month Plan at a Glance

Phase	Months	Primary focus	Exit criterion — do not move on until this is true
Phase 0	Week 1	Resume surgery + baseline	Rewritten resume with real numbers; 10 baseline LeetCode problems attempted.
Phase 1	1 – 3	DSA foundations + Postgres/Redis/queues	150+ problems solved; Habex has Redis caching, rate limiting and a job queue running.
Phase 2	4 – 5	Cloud, CI/CD, observability + the capstone	Habex deployed on AWS via GitHub Actions with metrics and a published load-test result.
Phase 3	6 – 7	System design, LLD, machine coding, mocks	Fresh medium in 25 min out loud; 5 mock interviews done; 10 timed machine-coding builds.
Phase 4	8 – 10	Applications, interviews, negotiation	Three offers in hand within a 3-week window.

> **THE RULE THAT MAKES THIS WORK**
>
> **DSA runs every single day from week 1 to the last interview.** It is the long pole and the only truly non-negotiable track. Everything else is layered on top of it. Two problems a day for eight months is ~480 problems — far more than you need. Consistency beats intensity; a two-week gap costs you a month of re-learning.

PHASE 0

Resume Surgery & Baseline

Week 1 · ~8 hours total

Free wins. These cost one weekend and raise your callback rate before you have learned anything new.

STEP 0.1 — PUT A NUMBER ON EVERY BULLET

Not one figure appears in the current resume. You have the data; you simply did not write it down. Go find these and put them in:

- **PDU Gen-3:** how many units shipped / in the field? How many concurrent touch panels and admin sessions? The 5-second sampler over a one-hour window = how many datapoints held in memory, mirrored to disk how often? How many alarm types? What is the panel's resolution and frame budget? How large is the ARM64 binary and how long does the installer take end to end?
- **Backend:** 17 route modules across how many endpoints? p95 response time on the device? How many roles and permissions in the RBAC matrix?
- **Habex:** users, rows in the largest table, requests/day, 12 migrations, 3 edge functions.
- **Jourvia:** screens, Jest test count / coverage %, Firestore rules covering how many collections.

Where a count is genuinely small, use a *hard constraint* instead — latency, uptime, protocol timing, memory ceiling. "Held p95 under 120 ms on a 512 MB ARM board" is stronger than any user count.

STEP 0.2 — CUT THE SIGNAL-LOWERING MATERIAL

- **Remove Time Tracker** (Tkinter + Selenium). It reads as scripting, not engineering, and it is your weakest project sitting in prime page-two space.
- **Remove the Certifications line.** "SQL (CodeChef), Digital Skills (HackerRank)" actively lowers the bar you appear to clear. Keep NCC as a single clause.
- **Trim the skills block.** It currently lists roughly seventy technologies, which reads as padding. Cut to what you would defend in an interview. Tkinter, Selenium and MySQL can go.

STEP 0.3 — RETARGET THE SUMMARY

It currently opens with "industrial automation hardware", which tells a Zepto recruiter you are not their candidate. Reopen with the transferable identity and demote the hardware to context:

> *"Full stack engineer, 3 years, TypeScript end to end. Sole owner of a production platform — React operator UI, Node/Express API, real-time telemetry over Socket.IO, JWT + RBAC auth, and automated deployment — shipped on customer-facing industrial hardware. Comfortable from the browser down to the Linux service."*

STEP 0.4 — REORDER PROJECTS AND TAKE A BASELINE

Promote **Habex** to the top — it is your only publicly reachable, consumer-shaped product, and it becomes the capstone in Phase 2. Then sit down and attempt **10 LeetCode easy/medium problems untimed**. Do not study first. You need an honest baseline to measure Phase 1 against.

PHASE 1

DSA Foundations + Backend Depth

Months 1 – 3 · ~15 hrs/week

Track A — DSA (daily, non-negotiable)

This is the hard filter. Zepto, Swiggy, CRED, Razorpay and Groww all run two DSA rounds before a human reads about your PDU architecture. No amount of production depth substitutes for it.

Setting	Decision
Resource	**Use the curriculum you already built** at `skill\dsa\` — 23 pattern-based chapters in TypeScript with a curated 150-problem plan. It is already scoped to exactly this phase. Do not go shopping for Striver or NeetCode on top of it; collecting resources is procrastination that feels like progress.
Language	**TypeScript.** You will interview in it, and it keeps your daily language fluency compounding.
Volume	2 problems/day, 6 days/week → ~150 by end of Phase 1, ~280 by interview season.
The 30-minute rule	Stuck 30 min → read the editorial → understand it fully → **re-solve from a blank file the next day.** Grinding blind for two hours teaches nothing and burns months.
Revision	Every Sunday, re-solve 5 problems from 2+ weeks ago. Without this you will forget most of what you did.

TOPIC ORDER (DO NOT SHUFFLE — EACH BUILDS ON THE LAST)

1. **Arrays & hashing** — frequency maps, prefix sums, Kadane. *(2 wks)*
2. **Two pointers & sliding window** — the highest-yield pattern in Indian product interviews. *(1.5 wks)*
3. **Binary search** — including "search on the answer space", which is the version they actually ask. *(1 wk)*
4. **Stacks & queues** — monotonic stack, next-greater-element family. *(1 wk)*
5. **Linked lists** — reversal, cycle detection, merge. Mechanical; get it over with. *(1 wk)*
6. **Trees & BST** — all traversals, LCA, diameter, validate, serialize. *(2 wks)*
7. **Heaps & intervals** — top-K, merge-intervals, meeting rooms. *(1 wk)*
8. **Graphs** — BFS/DFS, topological sort, union-find, Dijkstra basics. *(2 wks)*
9. **DP** — only the classic twenty: knapsack, LIS, coin change, edit distance, house robber, grid paths. *(2 wks)*
10. **Recursion & backtracking** — subsets, permutations, n-queens. *(1 wk)*

> **PHASE 1 DSA CHECKPOINT**
>
> You can solve an unseen LeetCode **easy** in under 15 minutes, and an unseen **medium** in under 40, while narrating your approach out loud. If not, extend Phase 1 by three weeks rather than moving on — a weak DSA base collapses under interview pressure.

Track B — Backend depth (your real differentiator once closed)

This is the track that turns you from "frontend dev who touched Node" into "backend-credible engineer", which is exactly the gap between ₹22 L and ₹35 L. Do every item **against Habex**, not a toy — you need artefacts you can screenshot and discuss.

> **YOU ALREADY OWN THIS CURRICULUM — WORK IT, DON'T REBUILD IT**
>
> `skill\backend\` is 24 chapters on exactly this stack (Express / PostgreSQL / Prisma / Redis / BullMQ / Docker) and its chapter-22 capstone builds the Habex backend end to end. The modules below map onto it directly — treat this table as the *sequence and exit criteria* for that curriculum, not as a competing syllabus. Same for `skill\system-design\` in Track D.

Module	Time	What to actually learn and build
B1. Postgres for real	2 wks	`EXPLAIN ANALYZE` until you can read a plan; B-tree vs partial vs composite vs covering indexes; why an index is skipped; the N+1 problem; transactions and the four isolation levels; deadlocks; connection pooling and why serverless breaks it. **Build:** find your slowest Habex query, profile it, index it, record the before/after milliseconds.
B2. Redis	1 wk	Cache-aside, TTLs and the invalidation problem, token-bucket rate limiting, distributed locks (and why naive `SETNX` is unsafe), pub/sub, sorted sets for leaderboards. **Build:** per-user rate limiting + a cache layer on Habex's heaviest read path.
B3. Queues & async	1 wk	BullMQ on Redis. Producers/consumers, retries with exponential backoff, dead-letter queues, idempotent consumers, at-least-once vs exactly-once. **Build:** move Habex's recurring-expense generation and notifications onto a queue with a retry policy.
B4. Money semantics	1 wk	Idempotency keys, optimistic vs pessimistic locking, double-entry ledgers, audit trails, why you never store money as a float, reconciliation, webhook replay safety. **This is the single topic Razorpay, CRED and Groww probe hardest** — and it is cheap to learn. **Build:** an idempotency-key middleware on Habex writes.
B5. API & testing craft	1 wk	Pagination (cursor vs offset and why offset breaks), versioning, error contracts, request validation with Zod, structured logging with correlation IDs. Integration tests with Supertest against a real test database, not mocks.

PHASE 2

Cloud, DevOps & the Capstone

Months 4 – 5 · ~15 hrs/week

Track C — Cloud & operations

Good news: this is a **short hop for you specifically**. You already run nginx as a reverse proxy, write systemd units, script deployments in Bash, ship versioned OTA updates with rollback, and read journald logs. That *is* DevOps — it simply happens on an ARM board instead of EC2. This track is mostly a vocabulary translation plus three genuinely new tools.

Module	Time	What to learn and build
C1. Docker	1 wk	Dockerfile, layer caching, multi-stage builds, small base images, `docker compose` for API + Postgres + Redis locally. **Build:** containerise the Habex API; compose the whole stack with one command.
C2. AWS fundamentals	2 wks	EC2, security groups, VPC basics, RDS Postgres, ElastiCache, S3, IAM roles, CloudWatch, an Application Load Balancer, Route 53 + ACM for TLS. Stay in free tier where you can and set a billing alarm on day one. **Build:** Habex API on EC2 behind nginx and an ALB, talking to RDS and ElastiCache.
C3. CI/CD	1 wk	GitHub Actions: lint → typecheck → test → build image → push → deploy. Branch protection, secrets, environments, rollback. You already built OTA-with-rollback for the PDU — same idea, and say so in interviews.
C4. Observability	1 wk	Structured JSON logs with correlation IDs, a `/health` and `/metrics` endpoint, RED metrics (rate, errors, duration), p50/p95/p99 as a concept, alerting thresholds, Sentry for errors. Journald → CloudWatch is the same discipline in a new place.
C5. Load testing	3 days	k6 or Artillery. Establish a baseline, apply your Redis cache and index work, re-run, and **record the p95 before and after**. This produces the numbers your resume has never had.

The Capstone: "Habex at Scale"

> **WHY UPGRADE RATHER THAN START FRESH**
> A new side project restarts your credibility from zero. Habex is already live, on your own domain, with 12 migrations and 3 edge functions. Scaling *it* closes all four blocking gaps at once — distributed systems, cloud/DevOps, observability, and measurable numbers — on a product you can already speak about with authority. One project, four gaps.

Ship these seven things by the end of month 5:

1. **Containerised API on AWS** — Docker image on EC2 behind an ALB, RDS Postgres, ElastiCache Redis, TLS on a subdomain.
2. **Redis caching + rate limiting** — cache-aside on the heaviest read path with a documented invalidation strategy; per-user token-bucket limiting.
3. **BullMQ job queue** — recurring expenses, notifications and monthly rollups run asynchronously with retries and a dead-letter queue.
4. **Idempotency + audit trail** — every write carries an idempotency key; every privileged action lands in an immutable audit log.
5. **CI/CD pipeline** — GitHub Actions running tests and deploying on merge to main, with a documented rollback path.
6. **Observability** — structured logs with correlation IDs, a metrics endpoint, Sentry, and one alert that actually fires.
7. **A load-test report** — k6 run at 200 concurrent users, p50/p95/p99 before and after optimisation, written up in the README with a diagram.

Then write **one architecture document** in the repo — a request-flow diagram, the caching and invalidation strategy, the queue topology, and a short "what I would do at 100× traffic" section. That document is what you screen-share in a

system design round, and it is the single highest-leverage artefact in this entire plan.

Track D — System design & LLD (start month 4, 3× per week)

You already have a curriculum underway at `skill/system-design/`. Reweight it: at three years of experience, **LLD is roughly 70% of what you will be asked** and HLD is 30%. Most candidates over-prepare distributed HLD and then fail an object-modelling round.

LLD — BE FLUENT IN THESE TEN, OUT LOUD, ON A WHITEBOARD

Rate limiter (all four algorithms) · in-memory cache with LRU + TTL · URL shortener · Splitwise / expense splitting · parking lot · elevator · notification service with multiple channels · payment idempotency layer · tic-tac-toe / chess board model · logging library. Practise SOLID, composition over inheritance, and the strategy/factory/observer patterns *as they appear in these problems* — not as abstract theory.

HLD — THE CONCEPTS YOU MUST BE ABLE TO DRAW AND DEFEND

Load balancing (L4 vs L7) · caching layers and invalidation · DB replication and read replicas · sharding and partition keys · CAP and what you trade · consistent hashing · message queues and back-pressure · CDN · rate limiting at the edge · idempotency · eventual consistency · the outbox pattern. Then practise five full designs: Instagram feed, WhatsApp, Uber matching, a payment gateway, and a Zepto-style 10-minute delivery/inventory system.

> **PRACTISE OUT LOUD, WITH A TIMER**
> Silent reading does not transfer to a design round. Record yourself doing a 40-minute design and watch it back once. It is uncomfortable and it is the fastest single improvement available in this plan.

PHASE 3

Interview Craft

Months 6 – 7 · ~15 hrs/week

Track E — Machine coding (Zepto, Swiggy, Flipkart, Meesho)

A 90-minute round where you build a working feature from scratch. They grade working software first, then structure, then edge cases. Do **ten of these, timed, no interruptions**:

Infinite-scroll feed with a skeleton loader · autocomplete with debounce and request cancellation · nested comment thread with reply/collapse · kanban board with drag-and-drop · multi-step form with validation · data table with sort, filter and pagination · a poll widget with live results · an image carousel · a todo app with optimistic updates and rollback · a file-tree explorer.

Scoring habits: ship something working by minute 60, then improve. Extract components early. Handle loading, empty and error states — most candidates forget all three. Narrate your trade-offs as you type.

Track F — JavaScript & TypeScript depth

You claim TypeScript heavily on the resume, so expect it probed hard.

- **JS internals:** event loop, macro vs microtasks, closures, `this` binding, prototypal inheritance, hoisting and the temporal dead zone.
- **Implement from scratch:** `Promise.all` / `race` / `allSettled`, debounce, throttle, curry, deep clone, memoize, an event emitter, `map` / `filter` / `reduce`, a retry-with-backoff wrapper.
- **Advanced TS:** generics with constraints, conditional and mapped types, discriminated unions, `infer`, template literal types, `satisfies`, and type-safe API contracts shared between client and server.
- **React internals:** reconciliation and keys, when `memo` / `useMemo` backfires, virtualisation for long lists, Suspense and error boundaries, concurrent rendering, custom hooks, and why your Redux vs Zustand choice went the way it did.

Track G — Behavioural & narrative

At the ₹35 L band the behavioural round is a real filter, not a formality. Write out and rehearse six STAR stories from your actual work:

1. The hardest technical decision you made alone (sole ownership of PDU Gen-3 — pick one architectural call).
2. A production bug you shipped and how you found it.
3. A time you disagreed with a stakeholder or a spec.
4. Something you built that failed or was rewritten.
5. How you learn a new technology under deadline.
6. **Why you are leaving hardware for a product company** — the most important one. The answer is *"I want to build for scale and for millions of users, and to grow alongside a large engineering team"*, never *"my current job is bad"*. Never criticise Delta.

Mock interviews — the highest-ROI hours in the plan

Book **at least five** before applying: two DSA, two system design, one behavioural. Use Pramp / interviewing.io / peers in your network. A mock exposes in 45 minutes what 40 hours of solo study will not: that you go silent when stuck, or jump to code before clarifying, or cannot estimate complexity out loud.

Phase 4 — Applications, Interviews & Negotiation (Months 8 – 10)

Target list and realistic bands (3.5 yrs, SDE-2)

Tier	SDE-2 band	Companies
Reach Hardest bar, apply last	₹35 – 55 L	Google, Uber, Atlassian, Rubrik, Databricks, Sprinklr, Arcesium, Media.net, Salesforce
Primary targets Your stated list — the ₹35 L zone	₹28 – 45 L	CRED, Razorpay, Groww, Zepto, Swiggy, PhonePe, Postman, Zeta, Navi, Flipkart
Strong fit, less crowded Same money, better odds for you	₹25 – 38 L	**Juspay** (near-perfect match for your low-level + systems bent), Setu, Hasura, Meesho, Slice, Jupiter, Rapido, Porter, Dream11, Ather & Bounce (hardware+software crossover — your PDU work is a *superpower* there, not a liability)
Calibration only Month 6, to practise	₹18 – 26 L	Mid-size product firms and funded startups you would not accept. Burn your rust here, not on CRED.

> **SEQUENCING MATTERS — YOU GET ONE SHOT PER COMPANY**
> Failed loops carry a **6–12 month cooldown**. Interview the calibration tier first, then strong-fit, then primary targets, then reach — in that order. Do not apply to CRED in month 3 "just to see".

Referrals beat everything

- **Start now, not in month 8.** A referral roughly triples your callback rate, and cold applications from a hardware company are the ones recruiters filter out first.
- **Build in public from month 1.** Post the PDU architecture (nothing confidential), the Habex scale upgrade, and what you learn each month. Eight months of visible building makes referral requests easy instead of awkward.
- **Ask specifically:** a short message naming the exact role and one line on why you fit converts far better than "can you refer me".
- Reconnect with SIT alumni and ex-Delta colleagues now, while you want nothing from them.

Negotiation — where ₹28 L becomes ₹35 L

1. **Compress your timeline.** Apply everywhere inside a two-week window so offers land together. A single offer has almost no leverage; three offers in one week has enormous leverage.
2. **Never state your current CTC first.** "I'm targeting a market-rate SDE-2 package and I'm optimising for the role — what band is this position budgeted at?"
3. **Anchor above target.** Ask for ₹40–42 L when you want ₹35 L. Recruiters expect to negotiate; your first number is a ceiling, never a floor.
4. **Negotiate base hardest**, then joining bonus, then ESOP. Base compounds into every future raise and every future offer.
5. **Get the written breakup** before responding — base / variable / ESOP vesting schedule and cliff / joining bonus and its clawback period.
6. **Say it plainly and stop talking:** "I have a competing offer at ₹X. I'd prefer to join you — can you get to ₹Y?" Then be silent and let them respond.

4. Your Weekly Schedule

About **15–17 hours a week** alongside a full-time job. This is sustainable for eight months; 25 hours a week is not, and burning out in month three costs you the whole plan.

Day	Before work	Evening (1.5 – 2 hrs)	Hours
Mon	1 DSA problem	Backend / cloud module	2.5
Tue	1 DSA problem	Backend / cloud module	2.5
Wed	1 DSA problem	System design or LLD	2.5
Thu	1 DSA problem	Capstone build	2.5
Fri	1 DSA problem	*Rest — deliberately*	0.5
Sat	—	3 hrs capstone + 2 hrs DSA (4 problems)	5.0
Sun	—	Revision: re-solve 5 old problems, write one LinkedIn post, plan the week	2.0
Weekly total			**17.5**

> **TRACK IT OR IT WILL NOT HAPPEN**
>
> Keep one spreadsheet with four columns: *date, problems solved, hours studied, what shipped*. Review it every Sunday. Missing one day is normal; missing a week means you re-plan on the spot rather than quietly abandoning the schedule. You built a habit tracker — use Habex for this.

5. Master Checklist

PHASE 0 — WEEK 1

- [ ] Every resume bullet carries a number or a hard constraint
- [ ] Time Tracker project and the Certifications line removed; skills block trimmed
- [ ] Summary rewritten to lead with full-stack TypeScript, not industrial hardware
- [ ] Habex promoted to the first project slot
- [ ] 10 baseline LeetCode problems attempted and logged
- [ ] DSA resource chosen (one only) and tracking spreadsheet created

PHASE 1 — MONTHS 1 – 3

- [ ] 150+ DSA problems solved; unseen medium in under 40 minutes
- [ ] Postgres: can read an `EXPLAIN ANALYZE` plan; one Habex query profiled and indexed with before/after numbers
- [ ] Redis: cache-aside layer and per-user rate limiting live on Habex
- [ ] BullMQ queue running with retries and a dead-letter queue
- [ ] Idempotency-key middleware on all Habex writes
- [ ] Integration tests running against a real test database
- [ ] LinkedIn posting started — at least 6 posts published

PHASE 2 — MONTHS 4 – 5

- [ ] Habex API containerised; full stack runs via `docker compose`
- [ ] Deployed on AWS: EC2 + ALB + RDS + ElastiCache, TLS, billing alarm set
- [ ] GitHub Actions pipeline: test → build → deploy, with a rollback path
- [ ] Structured logs with correlation IDs, a metrics endpoint, Sentry, one working alert
- [ ] k6 load test at 200 concurrent users; p50/p95/p99 before and after, written up

- [ ] Architecture document with diagrams committed to the repo
- [ ] DSA total past 250; LLD practice started

PHASE 3 — MONTHS 6 – 7

- [ ] 10 timed machine-coding builds completed
- [ ] 10 LLD problems solved out loud on a whiteboard
- [ ] 5 full HLD designs practised end to end with a timer
- [ ] JS internals implemented from scratch; advanced TS comfortable
- [ ] 6 STAR stories written and rehearsed, including "why leave hardware"
- [ ] 5+ mock interviews completed
- [ ] Resume updated with the capstone and its real numbers
- [ ] Referral conversations opened at 8+ target companies

PHASE 4 — MONTHS 8 – 10

- [ ] 2 – 3 calibration interviews taken first
- [ ] All primary-target applications submitted inside a two-week window
- [ ] Every offer's written breakup obtained and annualised correctly
- [ ] 3 offers in hand within a 3-week window
- [ ] Negotiated with a real competing number on the table

6. The Six Ways This Plan Fails

Failure mode	The fix
Collecting resources instead of solving	One DSA sheet, one system-design course. Buying a third course is procrastination wearing a productivity costume.
Skipping DSA because the projects are more fun	Projects get you past the recruiter. DSA gets you past round one. You need both, and DSA is the one you will avoid.
Applying too early	A failed loop costs a 6–12 month cooldown at that company. Wait for month 8 on your primary targets.
Starting new projects instead of finishing Habex	One deeply scaled project beats four shallow ones. Ship the seven capstone items before you build anything else.
No numbers on the resume	This is why you get filtered before a human reads a word about your architecture. Fix it in week 1.
Burning out at 25 hrs/week in month 2	15 hours sustained for eight months beats 30 hours for six weeks followed by nothing. Take Friday off, on purpose.

> **THE BOTTOM LINE**
>
> Your depth is already ahead of your years — sole ownership of a shipped platform, real auth, real-time systems, and deployment automation down to the Linux service. What you are missing is **proof** (DSA), **scale** (cloud, caching, queues), and **visibility** (numbers and a network). Eight months of 15 focused hours a week closes all three. ₹35 LPA is not a stretch target from there — it is the expected outcome of doing this properly.

Prepared September 2026 · Compensation bands reflect the Indian product-company market for SDE-2 at 3–4 years and should be re-checked against levels.fyi before negotiating.